

PROJECT 01

PolicyGraph-RAG: A Temporal, Contradiction-Aware, and Evidence-Verified Retrieval-Augmented Generation Framework for Healthcare Insurance Navigation

1. Problem Statement

Health-insurance policies contain complex information regarding coverage, exclusions, waiting periods, co-payments, room-rent limits, sub-limits, eligibility conditions, network hospitals and claim procedures. This information is distributed across multiple documents, including policy wordings, customer information sheets, schedules, renewal notices and endorsements. Policy conditions may also change over time. Consequently, customers, insurance representatives and support teams may find it difficult to determine which clause is applicable to a specific policy, customer and date.

Conventional Retrieval-Augmented Generation (RAG) systems retrieve passages based primarily on semantic similarity. Although the retrieved passages may be related to the question, they may not represent the currently applicable policy condition. For example, a base policy may specify a room-rent limit of ₹5,000, while a later endorsement increases the limit to ₹7,500. A conventional RAG system may retrieve both clauses or provide the older value without recognising that it has been superseded.

Existing systems also have difficulty distinguishing between an actual contradiction and a legitimate policy amendment. They may use evidence from the wrong policy version, insurer, product, customer schedule or effective period. This can produce answers that appear well supported but are operationally incorrect.

The proposed system addresses this problem by introducing an applicability-aware RAG framework. Before using a retrieved clause, the system verifies its policy identity, effective period, document authority, applicable conditions and supersession status. The system then generates a traceable answer supported by the applicable evidence. If the required documents are unavailable or the conflict cannot be resolved reliably, it abstains and recommends human review.

The project is positioned as an informational decision-support system. It will not approve claims, guarantee cashless treatment, provide medical advice or replace the insurer's official decision.

2. Objectives in Detail

The primary objective is to design and evaluate a reliable RAG framework for answering questions about evolving health-insurance policies.

The specific objectives are:

1. **Develop a structured policy-evolution model:** Represent insurers, products, policy instances, document versions, clauses, schedules and endorsements. The model will also capture relationships such as “amends,” “replaces,” “restricts,” “extends” and “supersedes.”
2. **Implement bitemporal policy management:** Maintain both the period during which a clause is legally or operationally valid and the date on which the document was recorded in the system. This supports current, historical and retrospective policy questions.
3. **Develop hybrid information retrieval:** Combine keyword-based retrieval, semantic vector search, metadata filtering and reranking. This will improve the retrieval of relevant clauses while preserving policy identity, document type, version and effective-date information.
4. **Create an applicability and supersession resolver:** Apply deterministic rules to determine whether a retrieved clause belongs to the correct policy, was valid on the requested date, satisfies the relevant conditions and has not been replaced by a later authoritative clause.
5. **Classify policy conflicts:** Distinguish among consistent clauses, explicit amendments, temporal supersession, scope-based differences, authority-resolvable conflicts and unresolved contradictions.
6. **Generate evidence-grounded answers:** Produce answers containing the applicable value or decision, important conditions, effective date, document name, clause reference and any relevant amendment information.
7. **Verify generated claims:** Decompose the generated response into individual claims and verify each claim against applicable evidence. Claims will be classified as supported, superseded, wrong-policy, wrong-date, scope-mismatched, contradicted or insufficiently supported.
8. **Support controlled abstention:** Avoid answering when documents are missing, OCR quality is poor, effective dates are unclear, policy conditions are unknown or authoritative clauses conflict.
9. **Develop a benchmark dataset:** Create an expert-validated dataset containing policy versions, amendments, temporal questions, conflicting clauses and evidence-level annotations.
10. **Evaluate business and technical performance:** Compare the proposed framework with standard RAG, hybrid RAG, Graph-RAG, temporal RAG and generic claim-verification approaches.

3. Dataset: Collection, Generation and Relevance

The project will develop a specialised dataset called the **Policy Evolution Benchmark for Health-Insurance Navigation**, or **PolicyEvoBench**.

The target dataset will cover approximately 20–30 health-insurance products from at least five insurers. It will include base policy documents, customer information sheets, policy schedules, renewal documents and publicly available endorsements.

Approximately 250–400 clause instances and 600–900 question-answer cases will be prepared across important categories such as coverage, exclusions, waiting periods, room-rent limits, co-payments, eligibility, network restrictions and sub-limits.

Public documents will be collected from official insurer websites, the Insurance Regulatory and Development Authority of India portal and relevant government health-scheme portals. For every document, the project will record its source URL, insurer, product name, product identifier or UIN, document type, version, issue date, effective date, retrieval date and checksum. This ensures provenance and reproducibility.

Text will be extracted using PDF parsers and Optical Character Recognition where necessary. Tables, headings, clause numbers, monetary values, percentages, durations, dates and cross-references will be preserved. Documents will be divided into semantic units based on clauses and policy sections rather than arbitrary text lengths.

Because real public documents may not contain enough clearly traceable amendments, controlled synthetic endorsements will also be generated. These will modify real clause structures through operations such as changing a monetary limit, extending a waiting period, adding an exclusion, introducing an exception or replacing an earlier clause. Synthetic documents will use fictional identifiers and will be clearly labelled. Approximately 60–70% of the final evaluation set will be based on real documents, while 30–40% will consist of controlled synthetic policy-evolution cases.

Questions will include current-policy questions, historical questions, effective-date boundary cases, supersession cases, contradictory clauses, missing-document scenarios and insufficient-evidence cases. Minimal pairs will also be created, where the same question produces a different answer because the policy, date or customer condition has changed.

Two annotators will identify the relevant clause, applicable clause, superseded clause, valid period, conflict category, expected answer and required system action.

Disagreements will be resolved through domain-expert adjudication.

The dataset directly supports the objectives by enabling evaluation of retrieval accuracy, temporal reasoning, supersession resolution, conflict classification, claim verification and safe abstention. Splitting the dataset by insurer and product will also test whether the system can generalise to previously unseen policy structures.

4. Technical Stack

The system will be developed using **Python 3.11 or later**. PyMuPDF and pdfplumber will support PDF extraction, while Tesseract or PaddleOCR will process scanned documents. Camelot or Tabula will extract policy tables.

PostgreSQL will serve as the primary database. Its range types, relational constraints and JSON capabilities will support policy versions, temporal intervals, metadata and audit records. The pgvector extension may be used for vector storage.

Retrieval will combine BM25 keyword search through OpenSearch or Elasticsearch with semantic embeddings from Sentence Transformers. Reciprocal Rank Fusion will combine retrieval results, and a cross-encoder will rerank the candidate clauses.

The applicability resolver will be implemented using Python and SQL-based deterministic rules. A lightweight graph representation using NetworkX may model amendment and supersession relationships. Neo4j can be included as an experimental Graph-RAG baseline but is not essential to the core system.

A controlled large language model will generate structured, cited answers. Claim decomposition and natural-language inference models will support response verification. FastAPI will provide service endpoints, while Streamlit may be used for the demonstration interface. Docker, Git, DVC, MLflow and pytest will support reproducibility, experiment tracking and testing.

5. Expected Outcomes

The project is expected to deliver:

- A formal method for determining whether a policy clause is applicable to a particular policy, date and customer context.
- A bitemporal representation of policy versions, endorsements and supersession relationships.
- Higher applicable-clause retrieval accuracy than conventional and metadata-filtered RAG systems.
- A measurable reduction in answers based on outdated, superseded or wrong-policy evidence.
- Improved classification of contradictions, amendments and scope-based differences.
- An applicability-aware claim verifier capable of identifying unsupported or contextually invalid statements.
- Safer system behaviour through calibrated abstention and human-review recommendations.
- An expert-validated benchmark for future health-insurance RAG research.
- A reproducible prototype providing evidence-backed answers with clause-level traceability.
- A foundation for customer-service assistance, policy comparison, amendment tracking, broker support and compliance review.

